Azure BGP Oscillation & Route Leak Analysis

Analyze and resolve BGP oscillation and BGP route leaks in Azure Virtual WAN–style hub-and-spoke topologies (and similar cloud-managed BGP environments).

This skill trains an agent to:

- Detect preference cycles that cause BGP oscillation
- Identify valley-free violations that constitute route leaks
- Propose allowed, policy-level mitigations (routing intent, export policy, communities, UDR, ingress filtering)
- Reject prohibited fixes (disabling BGP, shutting down peering, removing connectivity)

The focus is cloud-correct reasoning, not on-prem router manipulation.

When to Use This Skill

Use this skill when a task involves:

- Azure Virtual WAN, hub-and-spoke BGP, ExpressRoute, or VPN gateways
- Repeated route flapping or unstable path selection
- Unexpected transit, leaked prefixes, or valley-free violations
- Choosing between routing intent, UDRs, or BGP policy fixes
- Evaluating whether a proposed "fix" is valid in Azure

Core Invariants (Must Never Be Violated)

An agent must internalize these constraints before reasoning:

- ■ BGP sessions between hubs **cannot** be administratively disabled by customers as it's owned by azure
- ■ Peering connections **cannot** be shut down as a fix as it break all other traffic running on the connections
- ■ Removing connectivity is **not** a valid solution as it break all other traffic running
- ■ Problems **must** be fixed using routing policy, not topology destruction

Any solution violating these rules is invalid.

Expected Inputs

Tasks using this skill typically provide small JSON files:

File	Meaning
`topology.json`	Directed BGP adjacency graph
`relationships.json`	Economic relationship per edge (provider, customer, peer)
`preferences.json`	Per-ASN preferred next hop (may cause oscillation)
`route.json`	Prefix and origin ASN
`route_leaks.json`	Evidence of invalid propagation
`possible_solutions.json`	Candidate fixes to classify

Reasoning Workflow (Executable Checklist)

Step 1 — Sanity-Check Inputs
- Every ASN referenced must exist in `topology.json`
- Relationship symmetry must hold:
- `provider(A→B)` ⇔ `customer(B→A)`
- `peer` must be symmetric
- If this fails, the input is invalid.

Step 2 — Detect BGP Oscillation (Preference Cycle)

Definition
BGP oscillation exists if ASes form a preference cycle, often between peers.
Detection Rule
1. Build a directed graph: `ASN → preferred next-hop ASN`
2. If the graph contains a cycle, oscillation is possible
3. A 2-node cycle is sufficient to conclude oscillation.
Example pseudocode:

```python
pref = {asn: prefer_via_asn, ...}
def find_cycle(start):
path = []
seen = {}
cur = start
while cur in pref:
if cur in seen:
return path[seen[cur]:] # cycle found
seen[cur] = len(path)
path.append(cur)
cur = pref[cur]
return None
```

Step 3 — Detect BGP Route Leak (Valley-Free Violation)

Valley-Free Rule

Learned from	May export to
Customer	Anyone
Peer	Customers only
Provider	Customers only

Leak Conditions
A route leak exists if either is true:
1. Route learned from a **provider** is exported to a **peer or provider**
2. Route learned from a **peer** is exported to a **peer or provider**

Fix Selection Logic (Ranked)

Tier 1 — Virtual WAN Routing Intent (Preferred)

Applies to:
- ✔ Oscillation
- ✔ Route leaks
Why it works:
- **Routing intent operates above BGP** — BGP still learns routes, but does not decide forwarding
- **Forwarding becomes deterministic and policy-driven** — Intent policy overrides BGP path selection
- **Decouples forwarding correctness from BGP stability** — Even if BGP oscillates, forwarding is stable
For oscillation:
- Breaks preference cycles by enforcing a single forwarding hierarchy
- Even if both hubs prefer each other's routes, intent policy ensures traffic follows one path
For route leaks:
- Prevents leaked peer routes from being used as transit
- When intent mandates hub-to-hub traffic goes through Virtual WAN (ASN 65001), leaked routes cannot be used

- Enforces valley-free routing by keeping provider routes in proper hierarchy
Agent reasoning:
If routing intent is available, recommend it first.

Tier 2 — Export / Route Policy (Protocol-Correct)

For oscillation:
- **Filter routes learned from a peer before re-advertising** — Removes one edge of the preference cycle
- **Why this works**: In a cycle where Hub A prefers routes via Hub B and vice versa, filtering breaks one "leg":
- If Hub A filters routes learned from Hub B before re-announcing, Hub B stops receiving routes via Hub A
- Hub B can no longer prefer the path through Hub A because it no longer exists
- The cycle collapses, routing stabilizes
Example:
If vhubvnet1 (ASN 65002) filters routes learned from vhubvnet2 (ASN 65003) before re-advertising, vhubvnet2 stops receiving routes via vhubvnet1, breaking the oscillation cycle.
For route leaks:
- **Enforce valley-free export rules** — Prevent announcing provider/peer-learned routes to peers/providers
- **Use communities** (e.g., `no-export`) where applicable
- **Ingress filtering** — Reject routes with invalid AS_PATH from peers
- **RPKI origin validation** — Cryptographically rejects BGP announcements from ASes that are not authorized to originate a given prefix, preventing many accidental and sub-prefix leaks from propagating
Limitation:
Does not control forwarding if multiple valid paths remain.

Tier 3 — User Defined Routes (UDR)

Applies to:
- ✔ Oscillation
- ✔ Route leaks
Purpose:
Authoritative, static routing mechanism in Azure that explicitly defines the next hop for network traffic based on destination IP prefixes, overriding Azure system routes and BGP-learned routes.
Routing Behavior:
Enforces deterministic forwarding independent of BGP decision processes. UDRs operate at the data plane layer and take precedence over dynamic BGP routes.
For oscillation:
- **Oscillation Neutralization** — Breaks the impact of BGP preference cycles by imposing a fixed forwarding path
- Even if vhubvnet1 and vhubvnet2 continue to flip-flop their route preferences, the UDR ensures traffic always goes to the same deterministic next hop
For route leaks:
- **Route Leak Mitigation** — Overrides leaked BGP routes by changing the effective next hop
- When a UDR specifies a next hop (e.g., prefer specific Virtual WAN hub), traffic cannot follow leaked peer routes even if BGP has learned them
- **Leaked Prefix Neutralization** — UDR's explicit next hop supersedes the leaked route's next hop, preventing unauthorized transit
Use when:
- Routing intent is unavailable
- Immediate containment is required
Trade-off:
UDR is a data-plane fix that "masks" the control-plane issue. BGP may continue to have problems, but forwarding is stabilized. Prefer policy fixes (routing intent, export controls) when available for cleaner architecture.

Prohibited Fixes (Must Be Rejected)

These solutions are **always invalid**:
| Proposed Fix | Reason |

|--------------|--------|
Disable BGP	Not customer-controllable
Disable peering	prohibited operation and cannot solve the issue
Shutdown gateways	Breaks SLA / shared control plane
Restart devices	Resets symptoms only

Required explanation:
Cloud providers separate policy control from connectivity existence to protect shared infrastructure and SLAs.
Why these are not allowed in Azure:
BGP sessions and peering connections in Azure (Virtual WAN, ExpressRoute, VPN Gateway) **cannot be administratively shut down or disabled** by customers. This is a fundamental architectural constraint:
1. **Shared control plane**: BGP and peering are part of Azure's provider-managed, SLA-backed control plane that operates at cloud scale.
2. **Availability guarantees**: Azure's connectivity SLAs depend on these sessions remaining active.
3. **Security boundaries**: Customers control routing **policy** (what routes are advertised/accepted) but not the existence of BGP sessions themselves.
4. **Operational scale**: Managing BGP session state for thousands of customers requires automation that manual shutdown would undermine.
Correct approach: Fix BGP issues through **policy changes** (route filters, preferences, export controls, communities) rather than disabling connectivity.

Common Pitfalls

- ■ **Timer tuning or dampening fixes oscillation** — False. These reduce symptoms but don't break preference cycles.
- ■ **Accepting fewer prefixes prevents route leaks** — False. Ingress filtering alone doesn't stop export of other leaked routes.
- ■ **Removing peers is a valid mitigation** — False. This is prohibited in Azure.
- ■ **Restarting gateways fixes root cause** — False. Only resets transient state.

All are false.

Output Expectations

A correct solution should:
1. Identify oscillation and/or route leak correctly
2. Explain why it occurs (preference cycle or valley-free violation)
3. Recommend allowed policy-level fixes
4. Explicitly reject prohibited fixes with reasoning

References

- RFC 4271 — Border Gateway Protocol 4 (BGP-4)
- Gao–Rexford model — Valley-free routing economics